

PDE — Full Project Briefing for Claude / AI Assistants

Paste this entire document as system/context when asking Claude (or any AI) for **architecture understanding**, **feature ideas**, or **technical enhancements** on the Policy Deployment Engine.

Repo: <https://github.com/Hardhat-Enterprises/Policy-Deployment-Engine>

Org: Hardhat Enterprises (Deakin Capstone)

Status (as of mid-2026): GCP-first policy library; AWS/Azure placeholders only

Provider pin: HashiCorp Google provider **7.37.0** (`scripts/auto_test/provider_version.txt`)

Policy engine: OPA / Rego (helpers target OPA ~1.2 / Rego v1)

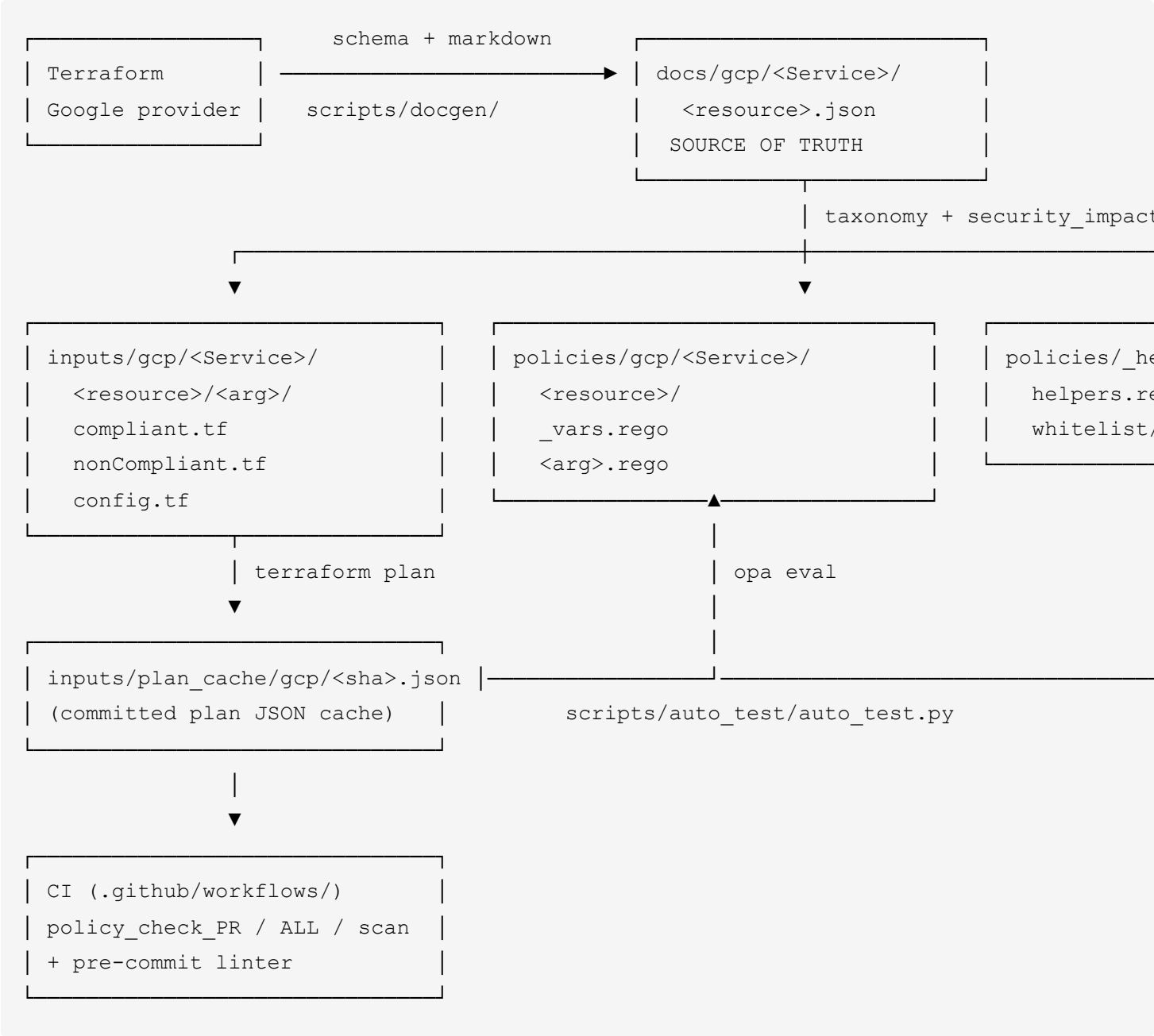
1. Purpose (why this exists)

PDE builds a **library of optional, platform-level security policies** for cloud Infrastructure-as-Code (Terraform).

- Target customer: an **enterprise with many teams/projects** (multi-tenant).
- Policies must be **generic and structural** — never hardcode team-specific names, keys, projects, emails, or resource IDs (region whitelist is the sanctioned exception).
- Goal: enterprises can adopt policies that make resources more secure by constraining Terraform configurations **before** apply (evaluated against `terraform plan` JSON via OPA).
- PDE is **not** a live cloud enforcement agent today; it is a **policy authoring + testing + CI gate** system that produces Rego policies + fixtures + documentation assessments.

Contributor model: Students are assigned Terraform resource types. They (1) assess every leaf argument in docs JSON for `security_impact`, then (2) write Rego + Terraform fixtures only for `true` args. Progress is tracked externally (learn.ybit / personal tracker) from `Service/...` branches.

2. High-level architecture



Three mirrored trees (linter enforces reconciliation to docs/):

Tree	Role
docs/	Every resource argument + human security_impact / rationale
inputs/	Per-argument Terraform fixtures (compliant vs non-compliant)
policies/	Per-argument Rego policies + shared helpers

Service folder names keep **spaces/parens** from provider taxonomy (e.g. `Compute Engine`, `Cloud Run (v2 API)`). Git branches use **slugs** (`compute_engine`, `cloud_run_v2_api`).

3. Core data contracts

3.1 Docs JSON (`docs/gcp/<Service>/<resource>.json`)

Generated/refreshed by `scripts/docgen/` from provider **schema** (not hand-written argument lists).

```
{
  "last_updated": "...",
  "provider_version": "7.37.0",
  "arguments": {
    "location": {
      "description": "...",
      "required": true,
      "type": "string",
      "security_impact": true,
      "rationale": "..."
    },
    "retention_policy": { "type": "block", "description": "...", "required": false },
    "retention_policy.is_locked": {
      "description": "...",
      "required": false,
      "type": "bool",
      "security_impact": "true/false",
      "rationale": ""
    }
  }
}
```

- **Leaf** args: have `security_impact` + `rationale` (filled by humans).
- **Block** args: containers only — no security fields; policies target dotted leaf children.
- Placeholder until assessed: `"security_impact": "true/false"`.

3.2 Assessment rules (`docs/ASSESSMENT_GUIDANCE.md`)

`security_impact: true` = worth a **generic platform policy**. Archetypes include:

- Disable public exposure
- Require encryption / CMEK (key non-blank, never pin a specific key)
- Secure protocol minimums
- Logging / monitoring
- Data residency (region whitelist — hardcoded AU regions OK as example)
- Prevent destructive loss (`deletion_policy=PREVENT` , etc.)
- Managed identity / credential hygiene
- Enable security features / disable weakeners

`false` for: identifiers, references to other resources, cosmetic fields, team addressing (CIDRs), free-form IAM `policy_data` , anything requiring team-specific hardcoded values.

3.3 Inputs fixtures

Per `true` argument:

```
inputs/gcp/<Service>/<resource>/<arg>/
  config.tf          # DO NOT EDIT — google provider block
  compliant.tf       # secure value(s); label compliant_example_N
  nonCompliant.tf    # insecure value(s); label non_compliant_example_N
```

Rules: only the tested resource type in the file (no real dependency resources — use fake IDs); include required args so `terraform plan` works.

3.4 Rego policies

```
policies/gcp/<Service>/<resource>/
  _vars.rego          # once per resource
  <arg>.rego           # one per true leaf (nested args: filename keeps dots;
                      # package suffix uses underscores)
```

Package pattern:

```
terraform.gcp.security.<service_slug>.<resource_type>.<arg_or_arg_with_underscores>
```

Example nested package end: `...advanced_options_config_json_parsing`

(file may still be `advanced_options_config.json_parsing.rego`)

`_vars.rego` fields:

- `friendly_resource_name` — human label in messages
- `resource_type` — Terraform type
- `resource_value_name` — attribute used to identify the instance (`name` , `rule_name` , `short_name` , ...)

Policy body uses shared helper:

```
conditions := [ [ {situation_description, remedies}, {condition, attribute_path, values,
result := helpers.get_multi_summary(conditions, vars.variables)
```

- Conditions in **one inner list** = AND
- Multiple outer scenario lists = OR (advanced)

Policy types (in `policies/_helpers/policies/`):

1. `whitelist`
2. `blacklist`
3. `range`
4. `pattern whitelist`
5. `pattern blacklist`
6. `element blacklist`

`attribute_path` walks plan `values` (nested blocks usually use index `0` , e.g.
`["match", 0, "src_network_context"]`).

4. Runtime / test pipeline

`scripts/auto_test/auto_test.py`

1. Pair each `<arg>.rego` with `inputs/.../<arg>/`
2. Hash fixtures + provider version → look up `inputs/plan_cache/gcp/<sha>.json`
3. Cache miss → `terraform plan` → JSON → write cache (must be committed)
4. `opa eval` policy against plan
5. Assert: flag `non_compliant_example_*` , do **not** flag `compliant_example_*`

Project-local provider mirror: `.terraform-cache/` (gitignored). First run may call `cache_setup.sh` .

Linters (`scripts/linters/`)

- Structure: docs ↔ inputs ↔ policies reconciliation
- Content: fixture naming, single resource type, Rego package suffix rules
- Branch name: `Service/<platform>/<service_slug>/<resource>` or `feature/<name>`
- Pre-commit only fails on **your changed** files (repo backlog doesn't block you)

CI (`.github/workflows/`)

Workflow	When	What
<code>policy_check_PR</code>	PRs touching docs/inputs/policies	Structural lint + content lint on changed files; for <code>Service/</code> PRs also doc completeness, policy coverage for all <code>true</code> args, OPA tests; labels <code>CI-Approved</code> / <code>CI-Review-Required</code>
<code>policy_check_ALL</code>	manual dispatch	Full-tree lint + full OPA suite
<code>policy-scan.yml</code>	related to personal tracker / branch scans	Used by contribution tracking

Important: PRs from forks are not accepted — work on Hardhat org branches.

5. Tooling map

Path	Purpose
<code>scripts/docgen/</code>	Generate/refresh docs JSON from provider schema
<code>scripts/linters/</code>	Taxonomy + content + branch checks
<code>scripts/auto_test/</code>	Plan cache + OPA harness
<code>scripts/folder_generator/</code>	GUI to scaffold inputs/policies from <code>templates/gcp</code>
<code>templates/gcp/</code>	<code>config.tf</code> , <code>compliant.tf</code> , <code>nonCompliant.tf</code> , <code>_vars.rego</code> , <code>policy.rego</code>
<code>Guide/Policy_writing_tutorial/</code>	Student how-to (some screenshots still stale — see <code>Guide/TODO.md</code>)

Path	Purpose
tests/_helpers/	Unit tests for shared Rego helpers

6. Contributor workflow (end-to-end)

1. Get assigned resource(s) on personal tracker
2. Checkout `Service/gcp/<service_slug>/<resource>`
3. **Docs phase (~50%):** for every leaf, set boolean `security_impact` + rationale; skip blocks; leave/use pre-filled `region / location` patterns
4. Request docs review / iterate on feedback
5. **Policy phase (~50%):** for each `true` arg → fixtures + Rego + `_vars` once
6. `python3 scripts/auto_test/auto_test.py "gcp/<Service>/<resource>"`
7. Commit (pre-commit) + push; open PR when ready
8. CI gate must pass for `Service/` PRs

Progress math: docs complete → ~50%; policies for all `true` args → ~100% for that resource. Writing policies before docs don't count toward the docs half.

7. Current scale (approx.)

- ~159 GCP service folders under `docs/gcp`
 - ~1277 resource JSON files
 - ~1029 policies / matching input argument dirs (order of magnitude; grows as contributors finish)
 - AWS/Azure: not implemented (placeholders only)
-

8. Known gaps & high-value technical enhancement directions

Use these when brainstorming HD / distinction features (policy writing alone is pass-level):

A. Product / deployment (beyond the library)

- **Policy pack packaging:** export selected policies as a versioned OPA bundle for enterprise CI
- **Terraform Cloud / Atlantis / GitHub Action consumer:** turn plan JSON → OPA → PR comment without students writing glue
- **Parameterized allowlists:** inject org-specific regions/CIDR exceptions via OPA data documents instead of hardcoded AU regions
- **Severity / category metadata** on policies (blocking vs advisory)

B. Authoring experience

- **Nested-attribute wizard:** generate `attribute_path` + package underscore name from docs dotted key
- **Fixture generator** from docs required fields + secure/insecure values
- **Docs review assistant:** already partially on learn.ybit — deeper consistency checks vs `ASSESSMENT_GUIDANCE.md`
- Refresh outdated Guide screenshots (`Guide/TODO.md`)

C. Quality / coverage analytics

- Dashboard: % docs assessed, % `true` args with policies, flaky policies, orphan plan_cache entries
- Detect contradictory assessments across similar resources (e.g. all `deletion_policy` should be true)
- Auto-suggest archetypes from argument descriptions

D. Engine / helpers

- New policy types (e.g. “must be non-empty string”, “HTTPS URL scheme only”) if repeatedly hand-rolled
- Better list/set semantics documentation (whitelist on list attributes)
- Stronger unit tests around AND/OR stacking and missing attributes

E. Multi-cloud

- Extend `docgen` + taxonomy + templates to AWS/Azure provider schemas
- Shared archetype layer so GCP/AWS policies share assessment language

F. CI / DX

- Faster PR feedback; smarter plan_cache pruning in maintainer workflows
- Better error messages when package name dots vs underscores mismatch
- Windows/macOS contributor path polish (partially done in auto_test cache setup)

G. Security of the system itself

- Supply-chain: pin OPA/Terraform versions, review plan_cache trust
 - Ensure fixtures never require real credentials
-

9. Design constraints (do not violate in proposals)

1. Policies must stay **generic** (no team-specific pins except region whitelist examples).
 2. `docs/` remains taxonomy source of truth; don't invent parallel schemas without migration.
 3. One policy ↔ one leaf argument (keep attribution clear).
 4. Fixtures: plan-only, no real cloud deps.
 5. Branch naming + no-fork PR rule for students.
 6. Prefer extending `_helpers` over copy-pasting Rego logic into every policy.
-

10. How to ask Claude useful follow-ups (examples)

After loading this briefing, ask things like:

- “Propose 3 HD-sized features that improve PDE for enterprise adoption, with MVP scope and touch points in this repo.”
- “Design parameterized region allowlists without breaking existing student policies.”
- “Sketch an architecture to run PDE policies as a GitHub Action on arbitrary Terraform PRs.”
- “Where should AWS support land first: docgen, templates, or helpers?”
- “Review my feature idea against section 9 constraints.”

When proposing code changes, reference concrete paths (`scripts/auto_test/` , `policies/_helpers/` , `docs/ASSESSMENT_GUIDANCE.md` , CI workflows) and keep student contributor UX intact.

11. My completed contribution context (optional — edit as needed)

I completed pass-level docs + policies for these Compute Engine resources on

`Service/gcp/compute_engine/...` branches:

- `google_compute_http_health_check` — `deletion_policy`
- `google_compute_https_health_check` — `deletion_policy`
- `google_compute_organization_security_policy` — `deletion_policy`,
`advanced_options_config.json_parsing`
- `google_compute_region_network_firewall_policy_rule` — 8 policies (`deletion_policy`,
`disabled`, `enable_logging`, `region`, `tls_inspect`, `match.src_network_context`,
`match.src_region_codes`, `match.src_threat_intelligences`); flipped `match.src_ip_ranges`
to false per review

I am targeting **HD**, so I need **technical enhancements beyond more policies**. Prefer feature ideas that strengthen the engine, CI, enterprise packaging, or multi-cloud path.

End of briefing. Prefer repository reality over older SharePoint/upskilling videos when they conflict.